

CodeLens

Product Requirements Document (PRD) & Technical Specification

1. Executive Summary

CodeLens is a highly performant, local-first developer tool designed to enable fast, intelligent, and structured search across codebases. By combining local file indexing (via SQLite FTS5) with an optional AI-driven semantic layer (via local models), CodeLens bridges the gap between basic keyword searches and advanced code comprehension, all without requiring a cloud connection or heavy IDE overhead.

2. Problem Statement & Objectives

Searching inside large codebases using standard tools or basic IDE text search is often slow, limited to exact keyword matches, and devoid of contextual understanding regarding how functions, classes, and files relate to one another.

Core Objective: Sub-100ms Search

The architecture prioritizes returning keyword and structured search results in under 100 milliseconds, minimizing context switching for developers.

- **Speed:** Achieve sub-100ms latency for keyword and structured searches.
- **Comprehension:** Understand code structures (functions, classes, variables) via Abstract Syntax Trees (AST) rather than just raw text.
- **Context:** Map call relationships to show where and how functions are utilized.
- **Intelligence:** Introduce semantic search capabilities to find logic conceptually, not just textually.
- **Privacy & Reliability:** Operate entirely locally (offline-first) to protect proprietary code and ensure zero latency from network calls.

3. Target Audience & Scope

Target Users

- **Professional Software Engineers:** Navigating large, unfamiliar, or legacy codebases.
- **Open-Source Contributors:** Needing to quickly understand project architecture.
- **Computer Science Students:** Analyzing complex repositories for learning.

Scope Boundaries

In Scope	Out of Scope (Non-Goals)
Local directory scanning and Git repository cloning.	Full IDE capabilities (debugging, compiling).
Parsing of major programming languages via AST.	Executing or running the code in a sandbox.
Local database indexing (SQLite FTS5) for instant retrieval.	Cloud-first synchronization or mandatory remote hosting.
Local AI embedding generation (Transformers.js) for semantic search.	API dependencies on external LLMs (e.g., OpenAI API).

4. System Architecture & Modules

The system follows a modular architecture to ensure components can be scaled, tested, or swapped independently. Data flows linearly: **User UI** → **API Server** → **Search/AI Layer** → **Index Engine** → **Local Storage**.

Module	Responsibilities	Proposed Tech Stack
1. Workspace Manager	Accept local directory paths, map workspace state.	Node.js (fs, path)
2. File Parser	Read file contents, filter out binaries. Extract AST structures.	Tree-sitter (AST)
3. Index Engine	Build and maintain inverted index. Handle incremental updates.	SQLite (FTS5)
4. Search Engine	Execute user queries. Rank results by match and structural importance.	Custom Logic + SQL
5. Code Intelligence	Map call relationships. Answer "where is this used?".	AST Traversal
6. AI Layer (Optional)	Generate vector embeddings for natural language semantic search.	Transformers.js
7. User Interface	Lightweight search bar, results list, syntax-highlighted preview.	React / Electron

5. Database Schema & Index Design (Deep Dive)

Using SQLite with the FTS5 extension pushes indexing into highly optimized C code, keeping memory usage low.

```
-- Table 1: File Metadata (For Incremental Indexing)
CREATE TABLE files (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  file_path TEXT UNIQUE NOT NULL,
  file_hash TEXT NOT NULL,
  last_indexed DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Table 2: The FTS5 Inverted Index
CREATE VIRTUAL TABLE search_index USING fts5(
  file_id UNINDEXED,      -- Foreign key to files.id
  entity_type UNINDEXED,  -- 'file', 'function', 'class'
  entity_name,            -- e.g., 'getUserData'
  content,                -- Code snippet body
  line_number UNINDEXED,
  tokenize='unicode61 tokenchars "_"'
);
```

The custom tokenizer tokenchars "_" ensures that snake_case variables in code are treated as single identifiable tokens rather than split into separate words.

6. Functional Requirements

- **Repository Ingestion:** Users must be able to point the application to an existing local folder to scan.
- **Instant Search:** Users must be able to type in a search bar and see live, ranked results updating instantly.
- **Filter & Facet:** Filter results by file type (e.g., .ts) or entity type (e.g., function definitions).
- **Code Preview:** Clicking a result displays a read-only, syntax-highlighted preview automatically scrolled to the match.
- **Semantic Queries:** Users can type natural language (e.g., "how is user authentication handled?") and receive relevant snippets via local embeddings.
- **Incremental Updates:** The system must watch directories (e.g., via chokidar) for file saves and update the index in the background.

7. Non-Functional Requirements

Resource Efficiency

The background indexer must utilize minimal RAM (< 150MB). Massive inverted indexes must be stored on disk via SQLite rather than loaded entirely into Node.js memory.

- **Resilience:** The parser must gracefully skip binary files, large minified bundles, or unsupported types without crashing.
- **Security:** Operates in a sandboxed environment with read-only access to user codebases to prevent accidental modification.

8. Phased Implementation Plan

Phase 1: The Core Engine (Speed & Stability)

Implement Workspace Manager for local directory scanning. Build basic Regex-based File Parser (to establish workflow). Integrate SQLite with FTS5 for the Index Engine. Develop the core UI (Search bar, basic list, code preview).

Phase 2: Structural Intelligence (Context & Accuracy)

Upgrade File Parser from Regex to Tree-sitter (AST) for accurate function/class extraction. Implement the Code Intelligence module for basic dependency and usage tracking. Integrate file system watchers for real-time incremental indexing.

Phase 3: The AI Layer & Advanced Features (Semantic & Visual)

Integrate Transformers.js to generate and store local embeddings. Enable semantic natural language search. Build visual dependency graph maps. Add multi-repository search capabilities.

9. Success Metrics

- **Search Latency:** 95th percentile of keyword searches complete in <100ms.
- **Index Build Time:** Capability to index a 10,000-file repository in under 60 seconds.
- **Memory Footprint:** Background process consumes less than 150MB of RAM during idle and search states.
- **User Accuracy:** High frequency of users clicking the first or second returned search result.